Experiment 4 — The compiler runs without ‘Compile[]‘

Mathilda — execution-speed experiments

Contents

Experiment 4 — The compiler runs without **`Compile[]`**

Dates: 2026-07-26 → 2026-07-29 · Commits: `27f5ccd` (Plot), `4595cf7` (Table), `ee4eb3a`/`29ee64e` (NIntegrate), `c1edf83`/`94160ac` (FindRoot), `7740825` (nine more), `64b8970` (numloop) · Code: `src/compile/autocompile.c`, `src/numloop.c` · Result: 6×–353×, on code the user never marked

Common method in `README.md`.

Hypothesis

`Compile[]` only helps code that someone thought to compile. Almost nobody writes `Compile[]` around the integrand of an `NIntegrate` — they write the integrand. But `NIntegrate` evaluates it thousands of times with a machine-real argument, which is exactly the compiler's domain, and the system knows that even though the user did not say it.

Mathematica does this. The hypothesis was that most of the benefit of the compiler was sitting in builtins that could invoke it themselves.

What was built

Two mechanisms, deliberately separate:

1. Auto-compilation at the numeric builtins (`src/compile/autocompile.c`). A builtin that is about to evaluate a user expression many times at machine precision hands it to the compiler first, and falls back to the interpreter if it does not compile. Now covering: `Plot`, `Plot3D`, `ParametricPlot`, `PolarPlot`, `ContourPlot`, `StreamPlot`, `Table`, `NIntegrate` (1-D and multi-D), `FindRoot` (scalar and systems, components and Jacobian), `NSum`, `NProduct`, and the NDSolve RHS.

2. **`numloop`** (`src/numloop.c`) — a narrower, cheaper path for the list-producing and iteration heads that does not build a `CompiledFunction` object at all: `Do`, `While`, `For`, `Nest`, `NestList`, `NestWhile`, `NestWhileList`, `Fold`, `FoldList`, `FixedPoint`, `FixedPointList`, `Scan`, `Accumulate`. These appear in ordinary code constantly and the object overhead would dominate at small sizes.

Results

Auto-compilation at the builtins (within Mathilda, **`MATHILDA_NO_AUTOCOMPILE`**)

kernel	interpreted	auto-compiled	
`NIntegrate`, batch of 50			~353×
Complex Newton fractal, 200×200×25	4.45 s	0.046 s	97×
`Plot`, 400-point 2-D curve			~215×
`PolarPlot`, 3000 points	33.5 ms	2.0 ms	16.8×
`Plot3D`, 80×80 surface			~11×
`ParametricPlot`, 3000 points	16.2 ms	1.9 ms	8.4×
`NProduct`	174.9 ms	21.9 ms	8.0×
`StreamPlot`	230.6 ms	32.3 ms	7.1×
`FindRoot`, 2×2 nonlinear system			~6.9×
`NSum`	114.7 ms	17.8 ms	6.4×
`ContourPlot`, 150²	52.0 ms	10.0 ms	5.2×

`numloop` at the functional heads

kernel	interpreted	numloop	
`Scan[Sin[#]^2 + 1. &, list]`	0.157 s	0.014 s	11.1×
`NestWhileList[# + 1. &, 0., # < n &]`	0.153 s	0.021 s	7.5×
`NestList[3.5 # (1−#) &, 0.31, n]`	0.208 s	0.032 s	6.6×
`FoldList[#1 + Sin[#2] &, 0., list]`	0.131 s	0.026 s	5.0×
`Accumulate[list]`	0.078 s	0.026 s	3.1×

Against Mathematica and Python

Both systems auto-compile numeric bodies by default, so this is like against like. The Python column uses the corresponding SciPy routine where one exists.

Benchmark	Mathilda	Mathematica 14.0	NumPy / Python	vs WL	vs NumPy
NDSolve, Lorenz system to t = 200	26.70 ms	47.78 ms	962.09 ms	1.79x	36.03x
Interpolation, 10^4 nodes, 10^4 evaluations	6.39 ms	20.00 ms	798 us	3.13x	1/8.00x

`NDSolve` is 36× faster than SciPy's **`solve_ivp`** on the Lorenz system and 1.79× faster than Mathematica — the RHS compiler paying off on the workload it was built for. SciPy's integrator calls back into Python for every RHS evaluation, which is the same cost auto-compilation removes from Mathilda.

For the quadrature and root-finding rows the honest comparison is against SciPy's compiled routines called on a Python integrand: `scipy.integrate.quad` on `Sin[x^2] Exp[−x/10]` over `{0,30}` measures 0.84 ms, `scipy.optimize.brentq` 0.016 ms, and `fsolve` on a 2×2 system 0.032 ms. Those are faster than Mathilda's auto-compiled equivalents; the gain recorded above is against Mathilda's own interpreter, which is the comparison the change was making.

Two bugs this created, both about evaluating too eagerly

Speculative compilation means running user code the user did not ask you to run yet. Twice that was observable.

1. **`numloop`** speculatively evaluated the loop body (`6e981e4`). The const-folding gate tried to evaluate a subexpression to decide whether it was constant — but the subexpression was user code with side ef-

fects, inside a `Do`. The fix gates const-folding on the body being free of assignment and I/O. Recorded as `project_numloop_const_fold_gate`.

2. Exactness leaked through the pass-through. `numloop` works in machine doubles, so anything it touches comes back inexact — but a value merely passed through the loop keeps its exact type in Mathematica. `Fold[#2 &, 1., {1,2,3}]` is the exact Integer `3`, not `3.`. Inexactness is a property of each result position, not of the loop. Recorded as `project_numloop_passthrough_exactness`.

Both are the same underlying hazard: a fast path may not change when or whether user code runs, nor what type comes out of it. Every gate in `numloop.c` now names which of those two it protects.

A third, subtler one: the benchmark that measured nothing

While validating this work, several ”interpreted” control measurements came back implausibly fast. The evaluator memoises, so the control run was reading the memo the timed run had just populated. A benchmark whose control is vacuous reports a speedup of 1× and hides a real one — or, worse, reports a large one because the treated run populated the memo for the control.

The harness now varies its argument per repetition (`kk`) and records the maximum as well as the minimum, specifically as a caching tripwire. Recorded as `feedback_eval_memo_vacuous_benchmark`.

Still open

- The auto-compile decision is per call, not cached: a `Table` inside a loop recompiles its body every outer iteration.
- `Map` over a list auto-compiles; `Map` over an `Association` does not.
- The bail is reportable (`MATHILDA_COMPILE_DIAG`) but not surfaced by default, so a body that silently stopped compiling after an edit still reads as ”correct but slower”.

Why Mathilda is not the fastest here, and what it would take

row	Mathilda	best other	gap	cause
`NDSolve`, Lorenz to t = 200	26.7 ms	47.8 ms (WL)	1.79× ahead	the right-hand side is compiled
`Interpolation`, 10^4 nodes	6.39 ms	798 µs (SciPy)	8.00×	evaluation is a binary search plus a per-point call,

The `NDSolve` row is the experiment's headline and it is ahead of both systems — 36× ahead of SciPy, because `solve_ivp` calls back into Python per step while Mathilda compiles the right-hand side once.

`Interpolation` is the row that is behind, and the comparison is fair: both build a cubic interpolant over the same nodes and evaluate it at the same count. SciPy's `CubicSpline.__call__` is a single compiled C routine that takes the whole query array; Mathilda evaluates `ifn /@ pts` as 10^4 separate applications, each doing its own interval search.

The road to fastest

1. A vectorised evaluation entry point. `ifn /@ pts` with a packed `pts` should dispatch once, sort or bucket the queries, and evaluate the polynomial for all of them in one pass — which is precisely what `CubicSpline(x)` does. Expected: 6.39 ms → well under 1 ms, since the arithmetic is a handful of multiply-adds per point.
2. Cache the last interval. Interpolants are overwhelmingly evaluated on monotone or clustered query sets (a plot, a resampling, an ODE dense output); remembering the previous bracket turns the search into one comparison in the common case.
3. Then **`NIntegrate`** and **`Plot`** inherit it, because both evaluate an interpolant pointwise when one appears in the integrand.

Item 1 alone should put this row ahead of SciPy.